OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

# PROJEKT REPORT

Hamed Ramezanzadeh

## Digital Design of a CNN Accelerator on ASIC

A Complete RTL-to-GDSII Flow on the IHP SG13G2 130 nm Technology

June 26, 2026

| | |
|---|---|
| Faculty: | Electrical and Information Technology |
| Study Programme: | M.Sc. Electrical and Microsystems Engineering |
| Deadline: | June 2026 |
| Supervisor: | Prof. Dr.-Ing Florian Aschauer |

# Contents

# 1 Introduction

## 1.1 Motivation and Goal

Convolutional Neural Networks (CNNs) require large amounts of computation. Running them on a general-purpose processor is slow and power-consuming, which is why dedicated hardware accelerators are used to perform the required operations far more efficiently [8]. This project designs such a CNN accelerator as an Application-Specific Integrated Circuit (ASIC), using the open-source IHP SG13G2 130 nm Process Design Kit [1] and the Cadence digital design toolchain.

The primary goal is to execute and document the **complete RTL-to-GDSII design flow**: from a hardware description in SystemVerilog, through logic synthesis and gate-level verification, to a fully placed-and-routed layout that passes Design Rule Checking (DRC) and is exported as a manufacturable GDSII file. The emphasis is on performing each stage of a real ASIC flow rather than on maximising inference performance. A secondary goal is to implement a CNN that exercises all the essential hardware building blocks of a real accelerator: convolution, activation, memory buffering, and pooling.

## 1.2 Network Architecture

The accelerator implements a fixed two-layer CNN operating on 32×32 single-channel (grayscale-format) input data, structured as 8-bit unsigned pixel values. For verification purposes, the input is generated by the testbench as a deterministic ramp pattern (pixel[i] = i mod 256), rather than a real captured image, since the project scope explicitly excludes real image datasets (see Section 1.1). The architecture, locked early in the project, is summarised in Table 1.

| Stage | Operation | Output | Weights |
|---|---|---|---|
| Input | 32x32x1 grayscale (uint8) | 32x32x1 | – |
| Conv1 | 3x3, stride 1, pad 1, 4 ch + ReLU | 32x32x4 | 36 B |
| Conv2 | 3x3, stride 1, pad 1, 8 ch + ReLU | 32x32x8 | 288 B |
| MaxPool | 2x2 window, stride 2 | 16x16x8 | – |

Table 1: Fixed CNN architecture implemented by the accelerator.

The following are explicitly out of scope: network training, real image datasets, classification-accuracy measurement, external (DRAM) memory, and a host bus interface. All 324 bytes of weights are stored as hardcoded constants, and correctness is verified against a Python golden reference model.

## 1.3 Tools and Technology

The project uses the full Cadence digital design suite on the university Linux server, together with the open-source IHP SG13G2 PDK (Table 2).

| Item | Details |
|---|---|
| Cadence Xcelium 24.03 | RTL simulation and gate-level simulation [4] |
| Cadence Genus 23.11 | Logic synthesis (run: 25 May 2026) [5] |
| Cadence Innovus 23.31 | Place and route, CTS, DRC, GDSII (run: 7 Jun 2026) [6] |
| IHP SG13G2 PDK | 130 nm CMOS, open-source [1] [2] |
| Standard cell library | sg13g2_stdcell_typ_1p20V_25C |
| SRAM macro | RM_IHPSG13_1P_4096x8_c3_bm_bist |
| Server | ei-vm-018.othr.de |

Table 2: Tools and technology used in the project.

The starting point for the lab environment and flow scripts was the digital design lab exercise repository provided by the supervisor [3].

# 2 System Architecture

## 2.1 Data Representation and Memory

All arithmetic uses 8-bit fixed-point representation. Activations are unsigned 8-bit integers (0–255), weights are signed 8-bit integers (-128 to 127), and the accumulator is 32-bit signed to avoid overflow when summing up to 36 products. This INT8 scheme is the standard choice for efficient CNN inference hardware, because CNNs tolerate low-precision quantization with little accuracy loss [9]. The resulting feature-map sizes are listed in Table 3.

| Stage | Ch. | Spatial | Size | Note |
|---|---|---|---|---|
| Input image | 1 | 32x32 | 1 KB | |
| Conv1 output | 4 | 32x32 | 4 KB | |
| Conv2 output | 8 | 32x32 | 8 KB | peak |
| Pool output | 8 | 16x16 | 2 KB | |

Table 3: Feature-map sizes; the 8 KB Conv2 output sets the SRAM bank size.

## 2.2 Module Structure

The design is partitioned into seven SystemVerilog modules. The **control path** (controller_fsm and addr_gen) decides when operations happen; the **datapath** (conv_engine, weight_rom, pool_engine, and feature_sram) performs the computation. The cnn_top module is neither control nor datapath – it is the top-level interconnect that wires all sub-modules together and routes the ping-pong bank-select signals. This separation follows standard digital-design practice [10] and makes each module independently verifiable. The overall dataflow is illustrated in Figure 1, and the modules are summarised in Table 4.

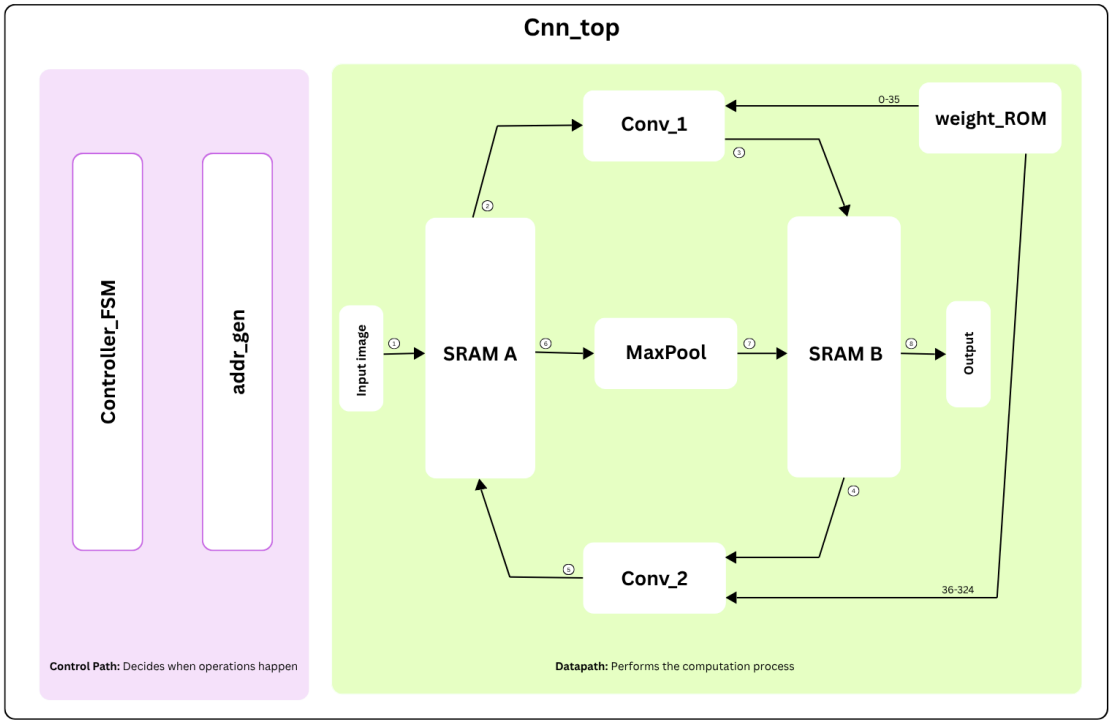


Figure 1: Simplified dataflow diagram of the cnn_top accelerator, intended to illustrate the ping-pong data movement between stages.

The control path (purple, left) consists of controller_fsm and addr_gen. The datapath (green, right) shows the data flow through the two SRAM banks, the convolution and pooling stages, and the weight_rom. Note that this diagram is a conceptual dataflow representation only and does not reflect the exact RTL module hierarchy; in particular, Conv1 and Conv2 are executed sequentially on the same conv_engine hardware instance, not on separate modules. The numbered arrows indicate the ping-pong sequence: ① input image loaded to Bank A, ② Bank A read by conv_engine (Conv1), ③ Conv1 result written to Bank B, ④ Bank B read by conv_engine (Conv2), ⑤ Conv2 result written to Bank A, ⑥ Bank A read by pool_engine, ⑦ pooled result written to Bank B, ⑧ final output read from Bank B.

| Module | Function |
|---|---|
| controller_fsm | Top-level Moore FSM: sequences IDLE -> CONV1 -> CONV2 -> POOL -> DONE and generates the ping-pong bank-swap signals. |
| addr_gen | Generates SRAM and weight-ROM addresses and the timing pulses (mac_valid, acc_clear, out_wr_en); handles zero-padding; uses CHW memory layout. |
| feature_sram | Wrapper around four IHP SRAM macros; implements ping-pong banks A and B (two macros, 8 KB each). |
| weight_rom | Combinational, zero-latency hardcoded ROM for all weights; four instances, one per MAC lane. |
| conv_engine | 4-lane parallel MAC datapath: four lanes each multiply one signed 8-bit weight (int8) by one unsigned 8-bit activation (uint8); the four products are summed via a Carry-Save Adder (CSA) tree into a 32-bit accumulator. The accumulator is cleared per output pixel (acc_clear) and accumulates across all tap groups. After the final group, the result passes through ReLU and 8-bit saturating truncation. The entire chain is purely combinational – making it the critical timing path of the design. |
| pool_engine | 2x2 MaxPooling FSM; for each output pixel cycles through six states (S_ADDR0–S_ADDR3, S_MAX, S_WRITE): reads one pixel per address state (single-port SRAM constraint), tracks the running maximum in S_MAX, and writes the result in S_WRITE. |
| cnn_top | Top-level interconnect (neither control nor datapath); wires all sub-modules together and routes the ping-pong bank-select signals from controller_fsm to feature_sram and the compute engines. |

Table 4: The seven RTL modules of the accelerator.

## 2.3 Ping-Pong Memory Architecture

Because the network is processed layer by layer, each layer's output must be stored while it serves as the next layer's input. Two SRAM banks (A and B), each 8 KB, alternate as source and destination (Table 5). This avoids overwriting data still in use.

| Stage | Reads from | Writes to |
|---|---|---|
| Conv1 | Bank A (input image) | Bank B |
| Conv2 | Bank B | Bank A |
| MaxPool | Bank A | Bank B |

Table 5: Ping-pong buffer assignment per pipeline stage.

Each bank is built from two RM_IHPSG13_1P_4096x8 macros (four macros total). These macros are **single-port** (a bank can be read or written in a cycle, but not both) and have a **one-cycle read latency** (data requested in cycle $N$ arrives in cycle $N + 1$). Both properties directly shaped the timing of addr_gen and were validated against the golden reference.

## 2.4 Dataflow and Cycle Budget

Output pixels are computed one at a time. For each output pixel, the addr_gen FSM passes through a fixed sequence of states: AG_NEWPIX (1 cycle) $\rightarrow$ AG_TAPS ($N$ cycles, one per MAC group) $\rightarrow$ AG_WRITE (1 cycle) $\rightarrow$ AG_NEXT (1 cycle), giving $N + 3$ clock cycles per output pixel. The 4-lane conv_engine processes kernel taps in groups of four: Conv1 needs $N = 3$ groups per pixel (one input channel, nine taps $\div$ four lanes), Conv2 needs $N = 9$ groups per pixel (four input channels, 36 taps $\div$ four lanes). The resulting clock cycles per pixel are $3 + 3 = 6$ for Conv1 and $9 + 3 = 12$ for Conv2. The full cycle budget is shown in Table 6.

| **Stage** | **Output pixels** | **MAC groups / pixel (N)** | **Clock cycles / pixel (N+3)** | **Total cycles** |
|---|---|---|---|---|
| Conv1 | 4,096 | 3 | 6 | 24,576 |
| Conv2 | 8,192 | 9 | 12 | 98,304 |
| MaxPool | 2,048 | – | 6 | 12,298 |
| **Total** | | | | **135,178** |

Table 6: Cycle budget for one complete inference pass. The "Clock cycles / pixel" column counts all addr_gen FSM states per output pixel; "MAC groups / pixel" counts only the AG_TAPS states in which mac_valid is asserted.

The derivations are: Conv1 $= 4096 \times 6 = 24,576$ and Conv2 $= 8192 \times 12 = 98,304$. Note that mac_valid is only asserted during AG_TAPS states, so the number of mac_valid pulses per layer is strictly less than the total clock cycles: $4096 \times 3 = 12,288$ pulses for Conv1 and $8192 \times 9 = 73,728$ pulses for Conv2 (verified in Section 3.1). The MaxPool count uses a different FSM (pool_engine), where each output pixel passes through six states – S_ADDR0, S_ADDR1, S_ADDR2, S_ADDR3, S_MAX, and S_WRITE – giving a base of $2048 \times 6 = 12,288$ cycles, plus approximately 10 overhead cycles for the S_IDLE $\rightarrow$ S_ADDR0 transition on pool_start and the S_DONE assertion, yielding 12,298. The total of 135,178 cycles was confirmed by simulation (week-17 testbench log [12]). Conv2 dominates at 73% of all cycles ($98,304/135,178$). At the 16 MHz maximum clock achievable with the unpipelined MAC (Section 5.2), one inference takes approximately $135,178/16,060,000 \approx 8.4$ ms.

# 3 RTL Design

This chapter summarises the role and key design points of each module.

## 3.1 Control Path: controller_fsm and addr_gen

The controller_fsm is a five-state Moore machine. It sequences the pipeline and generates the bank-swap signals, but performs no arithmetic. One important detail: the pool_done completion signal must be a **registered one-cycle pulse** rather than a combinational level, otherwise the FSM can misread "already done" at the start of a second run; the cause of Bug1 (Section 4.2).

The addr_gen module is the most complex control block. Its internal FSM implements the nested convolution loops entirely in hardware using five nested counters: output channel (oc), row (y), column (x), input channel (ic), and kernel tap (k), iterated in CHW order (channel-height-width). For each output pixel, the FSM steps through all tap groups sequentially, computing the SRAM read address for each activation as (ic×H×W + in_y×W + in_x), and the weight ROM index as (oc×Cin×9 + ic×9 + k) plus a layer offset of 36 for Conv2, to skip Conv1' s weights. Zero-padding is handled purely combinationally without storing a padded copy of the feature map: for each tap, addr_gen checks whether the input coordinates (in_y, in_x) fall within the valid 32×32 boundary. If they do, the SRAM read proceeds normally. If they fall outside (i.e., the tap lands on a border pixel requiring a padded zero), act_zero is asserted for that lane, the SRAM read is suppressed, and the MAC lane receives zero directly — all in the same cycle, with no memory access.

Verified pulse counts confirmed correctness: 12,288 mac_valid pulses for Conv1 and 73,728 for Conv2. These are the counts of AG_TAPS cycles only – the states in which mac_valid is asserted – and are smaller than the total clock cycles per layer (24,576 and 98,304 respectively, which include the AG_NEWPIX, AG_WRITE, and AG_NEXT states). These counts were verified in the addr_gen unit testbench using a `$display` counter that increments on every rising edge of mac_valid and compares the final total against the expected value at the end of each convolution phase, triggering `$error` on any mismatch. The expected values follow from the loop structure: $4096 \text{ pixels} \times 3 \text{ groups/pixel} = 12,288$ for Conv1 and $8192 \text{ pixels} \times 9 \text{ groups/pixel} = 73,728$ for Conv2.

## 3.2 Datapath: conv_engine, weight_rom, pool_engine

The conv_engine is the computational core. Four lanes each multiply an int8 weight by a uint8 activation; the four products are summed in a Carry-Save Adder (CSA) tree and added into a 32-bit accumulator. The accumulator is cleared once per output pixel and accumulates across all tap groups. After the final group, the result passes through inline ReLU and 8-bit saturation before being written back. Crucially, the whole chain – weight fetch, four multiplies, CSA tree, and 32-bit add – is **combinational with no pipeline registers**, executing in a single clock cycle. This is the critical timing path of the entire design (Section 5.2).

The weight_rom is purely combinational (zero latency), so weight data is available in the same cycle as the multiplication. Four instances feed the four MAC lanes in parallel. The pool_engine implements 2x2 MaxPooling: for each output pixel it cycles through six FSM states – four address/read states (S_ADDR0–S_ADDR3), one comparison state (S_MAX), and one write state (S_WRITE) – reading one pixel per address state (single-port SRAM constraint) while tracking the running maximum, then writing the result in S_WRITE.

## 3.3 Top-Level Integration

The cnn_top module connects all sub-modules and routes the ping-pong bank-select signals. During development, conv_engine was first replaced by an interface-compatible **stub** that produced fixed outputs, so the entire control path could be verified before the real datapath existed. This staged approach isolated integration risk. One subtlety surfaced here: synthesis implemented the conv_out_valid flip-flop using its inverted output, so the gate-level signal is active-low while the RTL signal is active-high. The testbench handles this automatically with a compile-time SYNTHESIS guard (see Section 4.4).

# 4 Verification

## 4.1 Strategy and Golden Reference

Verification used Cadence Xcelium at two levels. First, each module was checked individually with a focused unit testbench. Second, the full cnn_top was simulated with a known input image and its output was checked against a Python golden reference model.

The golden-reference approach used a two-step process. Three independent Python scripts (gr0_relu_clip.py, gr1_addressing.py, gr2_pixel_exact.py) each reconstruct, from first principles, the expected output value for one specific output pixel, computing the SRAM address sequence, applying the one-cycle synchronous read latency, performing the fixed-point MAC accumulation, and applying ReLU. Because these scripts are written from first principles rather than derived from the RTL, they provide a genuinely independent check. The values they produce (GR0=0, GR1=1, GR2=33) were embedded as localparam constants in tb_cnn_top.sv, and the testbench asserts the RTL-simulated output against these constants at runtime using a check_byte() task. A cross-check script (cross_check.py) subsequently parses the Xcelium log and automatically confirms that the RTL-reported values match the Python-computed ones, closing the verification loop without manual comparison.

The three pixels were chosen to exercise distinct correctness properties. GR0 (oc=0, y=1, x=1) targets ReLU clipping: all nine kernel taps are in-bounds, the MAC accumulation is negative, and the expected output after ReLU must be exactly zero. GR1 (oc=1, y=0, x=1) targets the SRAM read-latency and zero-padding interaction: three of the nine taps fall outside the image boundary (top-edge, in_y = −1), and the one-cycle registered SRAM latency shifts the effective data one group behind the driven address, making the correct expected output 1 rather than the 32 that an ideal zero-latency model would give. GR2 (oc=1, y=1, x=1) targets full in-bounds MAC arithmetic: all nine taps are valid, and the expected output of 33 was computed by tracing the same address-latency pipeline through the Python model.

The test input used throughout verification is a deterministic 32×32 ramp pattern generated directly in tb_cnn_top.sv, where each pixel value equals its linear memory index modulo 256. This pattern was chosen because it is fully reproducible by the Python golden reference model and exercises a wide range of pixel values (0–255) within a single deterministic test vector. Three significant bugs were found and fixed; they are described below as concrete illustrations of the verification process.

## 4.2 Bug 1: pool_engine FSM Restart (RTL)

Running two inferences back-to-back without a reset produced all-zero pooling output on the second run, while Conv1 and Conv2 remained correct. In the original implementation, pool_done was combinationally tied to the S_DONE state of pool_engine; meaning it remained HIGH as long as pool_engine stayed in S_DONE. When controller_fsm re-entered S_POOL for a second inference and pulsed pool_start, it sampled pool_done

= 1 on the very same cycle, immediately interpreted pooling as already complete, and skipped S_POOL entirely in a single cycle — so the second inference produced all-zero pooling output. The fix required two changes: (1) pool_done was converted to a registered one-cycle pulse, so it goes HIGH for exactly one cycle when S_DONE is entered and then returns LOW regardless of state; (2) pool_engine was forced to exit S_DONE immediately upon seeing pool_start HIGH, ensuring the FSM handshake is always a clean LOW→HIGH transition. Together, these guarantee that controller_fsm never sees a stale pool_done = 1 at the start of a new run.

### 4.3 Bug 2: addr_gen Conv2 Weight Offset (RTL)

Conv2 outputs were numerically wrong while Conv1 was correct. The weight-ROM address for Conv2 was missing the layer offset of 36 (the number of Conv1 weight bytes), so Conv2 used Conv1' s weights. Adding a single conditional offset term fixed the issue, and the output then matched the golden reference exactly.

### 4.4 Bug 3: conv_out_valid Polarity Inversion (GLS)

RTL simulation passed, but gate-level simulation produced all-zero output. Synthesis had implemented the conv_out_valid register using its inverted (Q_N) output as an area optimisation, making the gate-level signal active-low while the testbench expected active-high. The fix used a compile-time "`ifdef`" guard in the testbench. In RTL simulation, SYNTHESIS is not defined, so the testbench reads conv_out_valid directly (active-high). In GLS, the simulator is invoked with +define+SYNTHESIS, so the preprocessor selects the inverted version ( conv_out_valid), compensating for the Q_N mapping that Genus introduced. Concretely:

```
ifdef SYNTHESIS
      assign tb_conv_out_v = ~u_dut.conv_out_valid;  // Q_N: active-low
else
      assign tb_conv_out_v = u_dut.conv_out_valid;   // RTL: active-high
endif
```

Without this guard, GLS counted idle cycles ( 20,487 per run) instead of valid output cycles (4,096), because the inverted signal was HIGH during idle and LOW during valid — producing all-zero pixel output. The broader lesson is that synthesis may legally invert internal signal polarities, which is precisely why gate-level simulation is a required step rather than a formality.

### 4.5 Verification Results

All seven test categories passed after the fixes (Table 7). The gate-level pass confirms that synthesis introduced no functional change: the netlist is functionally equivalent to the RTL, matching the golden reference across all 2,048 output values.

The content of each test category is as follows. The addr_gen unit testbench runs Conv1 and Conv2 full-layer passes and verifies three counters against expected values: out_wr_en pulse count (4,096 for Conv1, 8,192 for Conv2), acc_clear pulse count (one

per output pixel), and mac_valid pulse count (3 per pixel for Conv1, 9 for Conv2); it also checks that layer_done is asserted exactly once and validates the SRAM address generated for the first output pixel of Conv1. The conv_engine unit testbench drives known activation and weight vectors through the MAC and verifies the output against a hand-computed expected_relu_sat() function, covering: single-tap all-ones (sum=4), saturation (300⟶255), ReLU clipping (negative⟶0), multi-tap accumulation across three groups, and negative weight handling. The pool_engine unit testbench runs seven named patterns (Gradient, AllZero, AllMax, PeakAtBR, PeakAtTL, ChannelIsolation, Reset_Recovery) and verifies all 2,048 output pixels against precomputed expected values derived from the Python golden reference (GR2). The GR1 system test checks three specific pixels at the integration level: GR0 (oc=0,y=1,x=1: all taps in-bounds, MAC=−66, ReLU⟶0), GR1 (oc=1,y=0,x=1: top-edge out-of-bounds tap, SRAM latency⟶1), and GR2 (oc=1,y=1,x=1: full 9-tap MAC=+33, ReLU⟶33). The GR2 system test compares all 2,048 pooled output pixels against the Python golden reference. The two-run test repeats the full inference without reset and verifies that RUN2 produces identical pixel counts and golden reference values to RUN1.

| **Test** | **Level** | **Tool** | **Result** |
|---|---|---|---|
| Unit: addr_gen (addresses, pulses, padding) | RTL | Xcelium | PASS |
| Unit: conv_engine (MAC, ReLU, saturation) | RTL | Xcelium | PASS |
| Unit: pool_engine (2x2 max, FSM) | RTL | Xcelium | PASS[1] |
| System: GR1 (addressing + ping-pong) | RTL | Xcelium | PASS |
| System: GR2 (full output, pixel-exact, 2,048 values) | RTL | Xcelium | PASS[2] |
| System: two runs without reset | RTL | Xcelium | PASS[1] |
| Gate-level simulation (full pipeline) | Netlist | Xcelium | PASS[3] |

Table 7: Verification results. [1]after Bug 1 fix, [2]after Bug 2 fix, [3]after Bug 3 fix.

# 5 Synthesis

## 5.1 Setup

The final logic synthesis was run with Cadence Genus 23.11 [5] on 25 May 2026, reading the seven SystemVerilog modules and the IHP standard-cell and SRAM liberty files (typical corner, 1.20 V, 25 degC). Synthesis effort was medium for all three passes (generic, mapping, optimisation), with a target clock period of 10 ns (100 MHz). Total runtime was 154.6 s.

## 5.2 Timing Results

The critical path runs entirely through the unpipelined MAC datapath, from the tap-group counter in addr_gen, through the weight ROM and the CSA tree, to the accumulator register. The key figures from the timing report are listed in Table 8.

| Metric | Value | Note |
|---|---|---|
| Target clock period | 10,000 ps | 100 MHz |
| Data-path delay | 62,274 ps | worst path |
| Setup + uncertainty | 299 ps | 199 + 100 ps |
| Required time | 9,701 ps | |
| **WNS (worst slack)** | **−52,573 ps** | setup, violated |
| TNS | −3,060,878 ps | 199 violating paths |
| Hold violations | none | hold not critical here |

Table 8: Synthesis timing summary (Genus, typical corner, 100 MHz target).

A Worst Negative Slack (WNS) of −52,573 ps means the critical path takes about 62.3 ns – roughly 6.2x the 10 ns target. The maximum operating frequency follows directly from the critical path delay: $f_{\max} = 1/62.274\ \text{ns} \approx 16.06\ \text{MHz}$, which is reported as approximately 16 MHz throughout this document. This is not a target but a measured ceiling imposed by the combinational depth of the MAC datapath. Closing timing at 100 MHz would require splitting the MAC into about seven pipeline stages (Section 7).

The 62.3 ns critical path starts at the tap-group counter register in addr_gen (`u_ag_tap_grp_reg[0]/CLK`) and ends at bit 31 of the accumulator register (`u_conv_engine_acc_reg_reg[31]/D`). It decomposes into four segments, as read from the Genus timing report: approximately 0.02 ns through address decode logic in addr_gen; approximately 5.3 ns through the weight ROM combinational decode (`g_weight_rom[3]`); approximately 56.5 ns through the CSA/Wallace tree and final ripple-carry adder generated by Genus's Datapath engine (`ADD_TC_OP_Y_final_adder`); and approximately 0.5 ns for the accumulator register setup time. The CSA tree and final adder together account for roughly 91% of the total path delay.

## 5.3 Area and Cell Count

From the area and QoR reports [13], the synthesised design contains 4,906 leaf instances (138 sequential, 4,768 combinational). The area is overwhelmingly dominated by the two SRAM banks, as expected for a memory-centric accelerator (Table 9).

| Instance | Cell area | Cells | Share |
|---|---|---|---|
| cnn_top (total) | 660,659 | 4,906 | 100% |
| u_sram_a (Bank A) | 293,253 | 28 | 44.4% |
| u_sram_b (Bank B) | 293,253 | 28 | 44.4% |
| weight_rom (x4) | 3,908 | 285 | 0.6% |
| controller_fsm | 1,081 | 63 | 0.2% |
| Remaining logic | 69,164 | 4,400 | 10.5% |

Table 9: Area breakdown from the Genus area report. Genus internal area unit.

The two SRAM banks together account for 88.8% of total cell area. The 138 sequential cells correspond exactly to the 138 clock sinks later balanced by clock tree synthesis.

## 5.4 Power

A vectorless power estimate (100 MHz, typical corner) gives a total of **23.74 mW** (Table 10). Switching power dominates at 68% of total (16.2 mW), of which logic accounts for 15.3 mW and registers 0.78 mW. Internal (short-circuit) power contributes 32% (7.58 mW), dominated by the SRAM at 5.10 mW. Leakage is negligible at 3.0 uW. These figures assume vectorless activity at 100 MHz; real operation at 16 MHz would reduce dynamic power roughly in proportion.

| Category | Leakage | Internal | Switching | Total |
|---|---|---|---|---|
| Memory | 1.6 uW | 5.10 mW | 0.04 mW | 5.14 mW |
| Register | 0.1 uW | 1.37 mW | 0.78 mW | 2.15 mW |
| Logic | 1.4 uW | 1.11 mW | 15.3 mW | 16.4 mW |
| Clock | 0 | 0 | 0.06 mW | 0.06 mW |
| **Total** | 3.0 uW | 7.58 mW | 16.2 mW | **23.74 mW** |

Table 10: Vectorless power summary (Genus, 100 MHz, typical corner).

# 6 Place and Route

## 6.1 Setup and Floorplan

Place and route used Cadence Innovus 23.31 [6], taking the Genus netlist, the SDC constraints, and the IHP LEF and timing files. A single timing view (typical corner) was used. Because the Quantus RC-extraction (QRC) file was not available in the lab, parasitics were estimated with Innovus's built-in extractor; all post-route timing therefore carries some uncertainty.

The four SRAM macros occupy the central-left region of the core, placed side by side horizontally: Bank A's two macros (u_sram_a/lower and u_sram_a/upper) on the left half, Bank B's two macros (u_sram_b/lower and u_sram_b/upper) on the right half. Standard cells are placed in the lower-right corner of the remaining area. Manual placement was necessary because Innovus's automatic macro placer produced arrangements where the two macros forming each bank were placed non-adjacently on opposite sides of the core, creating wide data buses that span the full chip width. The consequence was a non-zero routing overflow in the global route congestion report – horizontal and vertical overflow counts greater than zero, indicating the router could not find legal routes for all nets at that density. After switching to manual quadrant placement, the global route congestion report showed 0.00% overflow in both directions, confirming that the bus routing was resolved. The 0.00% overflow figure in Table 11 is therefore directly traceable to the manual placement decision. Floorplan and placement statistics are given in Table 11.

| Metric | Value | Note |
|---|---|---|
| core area | 453,727 $um^2$ | incl. all instances |
| Standard-cell area | 54,430 $um^2$ | 29,999 sites |
| Placement density | 9.81% | after place opt |
| Routing overflow | 0.00% H / 0.00% V | after global route |
| Total instances | 3,664 | in final GDS |
| Total nets | 38,759 | |
| Via instances | 23,759 | all layers |

Table 11: Floorplan and placement summary (Innovus).

The 10% placement density reflects the fact that the four SRAM macros physically occupy most of the core, leaving a small region for standard cells. Routing used six metal layers (Metal1–Metal5 plus TopMetal1).

The resulting floorplan is shown in Figure 2.

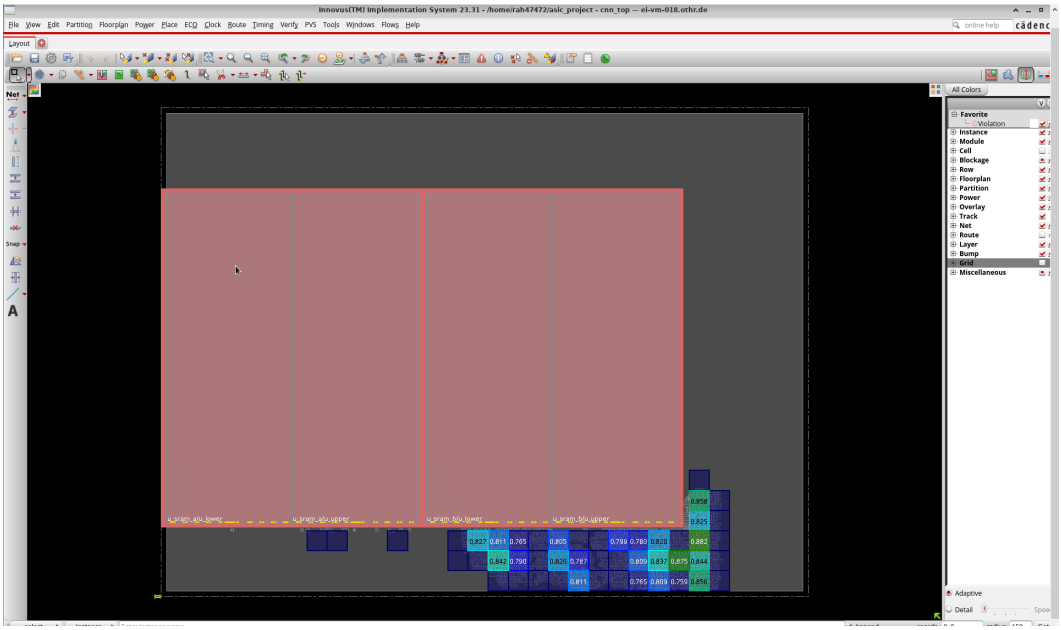

Figure 2: Innovus floorplan view of cnn_top. The four SRAM macros (u_sram_a/lower, u_sram_a/upper, u_sram_b/lower, u_sram_b/upper) occupy the central-left region of the core side by side, accounting for 88.8% of cell area. Standard cells are placed in the lower-right corner. Core area is 453,727 µm$^2$ with 9.81% standard-cell placement density.

## 6.2 Timing Closure

Setup timing closed to **0.000 ns** at the placement-optimisation stage (Table 13). The large improvement from the −52.6 ns synthesis WNS requires explanation. The SDC exported by Genus contains no `set_multicycle_path` declarations – only a `set_false_path` for the asynchronous reset. The improvement is the direct result of Innovus's `place_opt_design` command, which invokes the internal GigaOpt optimisation engine through a sequence of passes: initial placement, DRV fixing, global timing optimisation, area reclaiming, incremental replacement, and TNS fixing. Table 12 shows the WNS and TNS at each internal snapshot, as recorded in the `place_opt` log. The improvement is step-by-step and observable: WNS drops from −0.978 ns after initial placement to 0.000 ns after the TNS-fixing pass.

| **Snapshot** | **WNS (ns)** | **TNS (ns)** | **Density** |
|---|---|---|---|
| Initial placement | −0.978 | −16.220 | 12.00% |
| DRV fixing | −0.973 | −16.147 | 12.00% |
| Global opt (cell sizing) | −0.392 | −5.320 | 11.81% |
| Area reclaiming | −0.121 | −1.149 | 10.01% |
| Incremental replacement | −0.129 | −1.198 | 10.01% |
| TNS fixing | 0.000 | 0.000 | 10.04% |
| **Final summary** | **0.000** | **0.000** | **9.81%** |

Table 12: place_opt_design internal WNS/TNS progression (from Innovus log [13]).

It is important to note that this timing closure is not the same as physically closing at 100 MHz. The WNS values in Table 12 are computed from estimated wire RC parasitics (no QRC extraction), which systematically underestimate interconnect delay; the true post-layout delay would likely be higher. This is confirmed by the post-route WNS of –0.030 ns, which represents a more realistic estimate. The –0.030 ns is therefore the more credible figure, and even that carries uncertainty. After clock tree synthesis, hold timing was met with +0.001 ns slack and zero clock skew across all 138 sinks, requiring no hold buffers.

| Stage | Setup WNS | Hold WNS | Note |
|---|---|---|---|
| Placement opt. | 0.000 ns | – | see Table 12 |
| Post-CTS | – | +0.001 ns | 0 skew, 138 sinks |
| Post-route final | –0.030 ns | met | 1 path, est. RC |

Table 13: Timing closure across the P&R flow.

One net retained an unresolvable max-capacitance / max-fanout design-rule-value violation that the tool could not buffer automatically. The affected net is the global clock distribution net (`clk` tree root), which fans out to all 138 sequential sinks. The IHP SG13G2 standard-cell library defines a conservative max-fanout rule for this technology node; at 138 sinks, the pre-CTS driver violates this rule before the clock tree insertion phase adds its own buffers. Innovus's automatic fix attempts were blocked by the placement density constraint around the clock source cell. The violation is benign in practice – CTS subsequently balanced the tree to 0.000 ns skew across all 138 sinks – but it is recorded as a known limitation pending manual buffer insertion or a revised floorplan with a more central clock source location.

## 6.3 DRC and GDSII

DRC was run using the IHP SG13G2 rule deck integrated into Innovus (`verify_drc`). The check covered all six routing layers (Metal1–Metal5 plus TopMetal1) and both the standard-cell and SRAM-macro geometry. Table 14 shows the net count per metal layer; the routing is heavily concentrated on Metal2 (21,175 nets), which carries the majority of signal routing in horizontal tracks, while TopMetal1 carries only 14 nets (power strap connections).

| Layer | Net count | Primary use |
|---|---|---|
| Metal1 | 3,105 | local cell connections |
| Metal2 | 21,175 | main signal routing (horizontal) |
| Metal3 | 8,603 | signal routing (vertical) |
| Metal4 | 4,500 | longer-range routing |
| Metal5 | 1,362 | power/ground distribution |
| TopMetal1 | 14 | power strap connections |

Table 14: Net count per routing layer (Innovus post-route report [13]).

The final DRC run reported **zero violations** and **zero antenna violations** across all layers and all rule categories. No minimum-spacing, minimum-width, via-enclosure, or notch violations were found. The antenna check, which verifies that metal areas connected to gate oxides do not accumulate charge beyond the process limit during fabrication, also passed cleanly.

The final routed layout is shown in Figure 3.

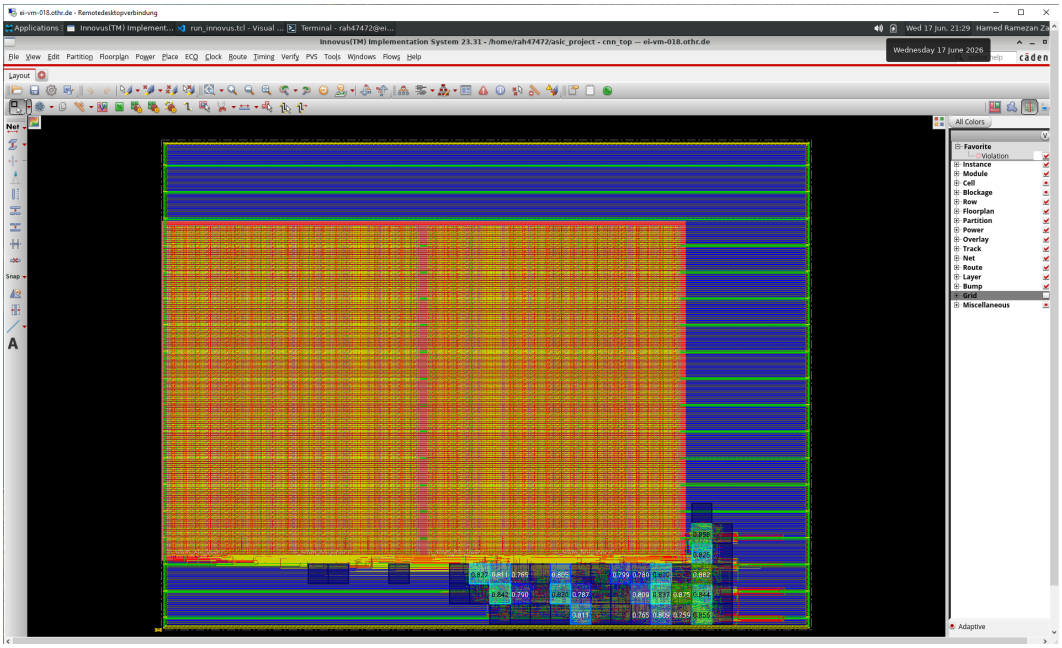

Figure 3: Post-route layout of cnn_top after detail routing across six metal layers (Metal1–Metal5 and TopMetal1). The dense horizontal routing tracks are visible across the SRAM macro region. Standard cells and their connections are visible in the lower-right corner. The layout passed DRC with zero violations and was exported as cnn_top_final.gds.

The GDSII file (`cnn\_top\_final.gds`, 3.1 MB) was exported using `streamOut` with the `-merge` option, which embeds the standard-cell GDS and all four SRAM-macro GDS files into a single merged output. This is required for the file to be self-contained and viewable in Virtuoso or KLayout without separate library references. The merged GDSII represents a complete, DRC-clean core implementation and is suitable for integration into a full-chip tapeout flow upon addition of an I/O pad ring and sealring.

# 7 Results and Discussion

Table 15 consolidates the key results of the complete flow.

| Metric | Value |
|---|---|
| Technology | IHP SG13G2, 130 nm CMOS |
| Target / achievable clock | 100 MHz target / 16 MHz achievable |
| Critical path (MAC) | 62.3 ns (WNS –52,573 ps at synthesis) |
| Leaf instances (Genus) | 4,906 (138 seq., 4,768 comb.) |
| SRAM area share | 88.8% |
| Total power (vectorless) | 23.74 mW (logic 16.4, SRAM 5.1) |
| Setup WNS (post-route) | –0.030 ns (1 path) |
| Hold WNS (post-CTS) | +0.001 ns (met) |
| Clock skew after CTS | 0.000 ns (138 sinks) |
| DRC / antenna violations | 0 / 0 |
| Core area | 453,727 $um^2$ |
| Total instances in GDS (Innovus) | 3,664 |
| GLS vs. golden reference | PASS – pixel-exact (2,048 values) |
| Inference latency | 135,178 cycles ( 8.4 ms at 16 MHz) |
| Final output | cnn_top_final.gds (DRC-clean) |

Table 15: Complete results summary.

The central outcome is that the entire RTL-to-GDSII flow was executed successfully on a real open-source PDK with industrial tools, with functional equivalence verified at the gate level and a DRC-clean layout. Three questions from the results table deserve explicit analytical treatment.

**Instance count: 4,906 (Genus) vs. 3,664 (Innovus GDS).** These two numbers come from different tools and counting conventions and are both correct. The 4,906 figure is the leaf-instance count from the Genus synthesis report: it counts every standard-cell primitive in the mapped netlist, including cells inside the SRAM macro wrappers that Genus can see. The 3,664 figure is the total-instance count from the Innovus layout database after place and route: Innovus treats each placed hard macro (here, each of the four SRAM tiles) as a single black-box instance regardless of internal cell count, and its own optimisation passes remove redundant buffers that Genus had inserted. The difference – approximately 1,242 instances – is therefore expected and does not indicate a discrepancy between the two flows.

**The SRAM area (88.8%):** The two 8 KB ping-pong banks are built from four RM_IHPSG13_1P_4096x8 hard macros, which are physically large structures optimised for density at the bit cell level. The combinational and sequential logic for all seven RTL modules combined occupies only 10.5% of cell area. This balance is inherent to a memory-centric accelerator: the network produces up to 8 KB of intermediate activations per layer, and those activations must be stored on-chip. Replacing the hard

macros with flip-flop arrays would have been far less area-efficient. The area profile is therefore not a weakness but an expected outcome of the design approach.

**Timing limitation to 16 MHz:** The 16 MHz ceiling is a direct consequence of keeping the MAC datapath fully combinational. As derived in Section 5.2, $f_{\max} = 1/62.274\ \text{ns} \approx 16\ \text{MHz}$. The 62.3 ns is dominated (91%) by the CSA/Wallace tree and final adder that Genus generates for the four-lane multiply-accumulate. This is a micro-architectural choice, not a PDK or layout limitation – the IHP SG13G2 library cells themselves can operate well above 100 MHz on shorter paths.

The most significant performance bottleneck, however, is not the clock rate – it is the serial channel access imposed by the single-port SRAM macro. Conv2 consumes 73% of inference cycles (98,304 of 135,178; $98304/135178 = 72.7\%$) because its four input channels must be read sequentially: with a 4-lane MAC but only one SRAM read port, the 36 taps per output pixel require nine sequential MAC groups. A dual-port macro or per-channel banked memory would allow parallel channel reads and could reduce Conv2 latency by roughly 4x, overshadowing the gain from pipelining the MAC alone.

# 8 Limitations and Future Work

The known limitations and their proposed remedies are summarised in Table 16. The most important is the unpipelined MAC; pipelining it into about seven stages is the single highest-impact improvement and the natural first step of any follow-on work.

| Limitation | Root cause | Proposed fix |
|---|---|---|
| MAC fails 100 MHz (62.3 ns path) | Entire multiply + CSA + accumulate is combinational | Pipeline into 7 stages; delay out_wr_en accordingly |
| Conv2 channel reads are serial | Single-port SRAM macro | Use dual-port or per-channel banked memory |
| Network cannot be changed | Weights hardcoded in ROM | Writable weight SRAM + AXI4-Lite load interface |
| No host connectivity | Scope excluded host interface | Add AXI4-Lite slave; enables RISC-V SoC integration |
| Post-route WNS −0.030 ns; 1 DRV net | Estimated RC (no QRC); high-fanout net | Obtain QRC deck; manual buffering of the net |

Table 16: Known limitations and proposed improvements.

These directions point naturally toward a follow-on master's thesis: integrating this accelerator with a RISC-V core as a complete System-on-Chip on the same IHP SG13G2 technology.

# 9 Conclusion

This project successfully completed the full digital ASIC design flow for a CNN accelerator on the IHP SG13G2 130 nm open-source PDK, using the Cadence Xcelium, Genus, and Innovus toolchain. The two-layer CNN was implemented with a 4-lane MAC datapath, ping-pong SRAM buffering, and a hardcoded weight ROM, and was verified pixel-by-pixel against a Python golden reference at both RTL and gate level. The synthesised design comprises 4,906 instances and consumes an estimated 23.74 mW, with the SRAM accounting for 88.8% of cell area. After place and route, hold timing is fully met, setup timing closes to –0.030 ns post-route (one path, estimated parasitics), and the final layout passes DRC with zero violations and was exported as a merged GDSII file.

Three real bugs – an FSM restart defect, a weight-addressing error, and a gate-level polarity inversion – were diagnosed and resolved, each reinforcing a key lesson about correctness and the gap between RTL and gate-level behaviour. The main limitation, the unpipelined MAC, is a conscious simplicity-versus-performance trade-off with a clear remedy. Overall, the project delivered a complete, verified, manufacturable design and, more importantly, hands-on mastery of synthesis-aware RTL design, verification, and physical implementation – a solid foundation for professional IC design work.

# List of Figures and Tables

# References

[1] IHP-GmbH, "IHP Open Source PDK – SG13G2 130 nm BiCMOS Technology," GitHub repository. Available: https://github.com/IHP-GmbH/IHP-Open-PDK

[2] IHP-GmbH, "SG13G2 Open Source Process Specification," IHP-Open-PDK documentation. Available: https://github.com/IHP-GmbH/IHP-Open-PDK

[3] F. Aschauer, "DD_Lab_exercise – Digital Design Lab Exercise," OTH Regensburg, GitHub repository. Available: https://github.com/baruaeee/DD_Lab_exercise

[4] Cadence Design Systems, "Xcelium Logic Simulator," User Guide, v24.03, 2024.

[5] Cadence Design Systems, "Genus Synthesis Solution," User Guide, v23.11, 2024.

[6] Cadence Design Systems, "Innovus Implementation System," User Guide, v23.31, 2024.

[7] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.

[8] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," Proceedings of the IEEE, vol. 105, no. 12, pp. 2295–2329, 2017.

[9] B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 2704–2713.

[10] N. H. E. Weste and D. M. Harris, CMOS VLSI Design: A Circuits and Systems Perspective, 4th ed. Boston, MA: Addison-Wesley, 2011.

[11] IEEE Standard for SystemVerilog – Unified Hardware Design, Specification, and Verification Language, IEEE Std 1800-2017, 2017.

[12] H. Ramezanzadeh, "Project-Thesis-CNN_ACC_on_ASIC," OTH Regensburg, GitHub repository (weekly progress reports, RTL source, synthesis and P&R reports). Available: https://github.com/HMDRZ69/Project-Thesis-CNN_ACC_on_ASIC

[13] H. Ramezanzadeh, Genus area report (`area.rpt`), Innovus place-and-route log, and post-route timing reports, in Project-Thesis-CNN_ACC_on_ASIC repository, dir. `Cadence/Genus/Final\_Genus\_Out/reports/` and `Cadence/Innovus/`. Available: https://github.com/HMDRZ69/Project-Thesis-CNN_ACC_on_ASIC

# Erklärung

1. Mir ist bekannt, dass dieses Exemplar der Projektbericht als Prüfungsleistung in das Eigentum der Ostbayerischen Technischen Hochschule Regensburg übergeht.
2. Ich erkläre hiermit, dass ich diese Projektbericht selbstständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

OTH Regensburg, 06/26/2026, Hamed Ramezanzadeh

---

Ort, Datum und Unterschrift

Presented by: Hamed Ramezanzadeh
Student ID: 3453962
Study Programme: M.Sc. Electrical and Microsystems Engineering
Time Frame: November 2025 – June 2026
Supervisor: Prof. Dr.-Ing Florian Aschauer